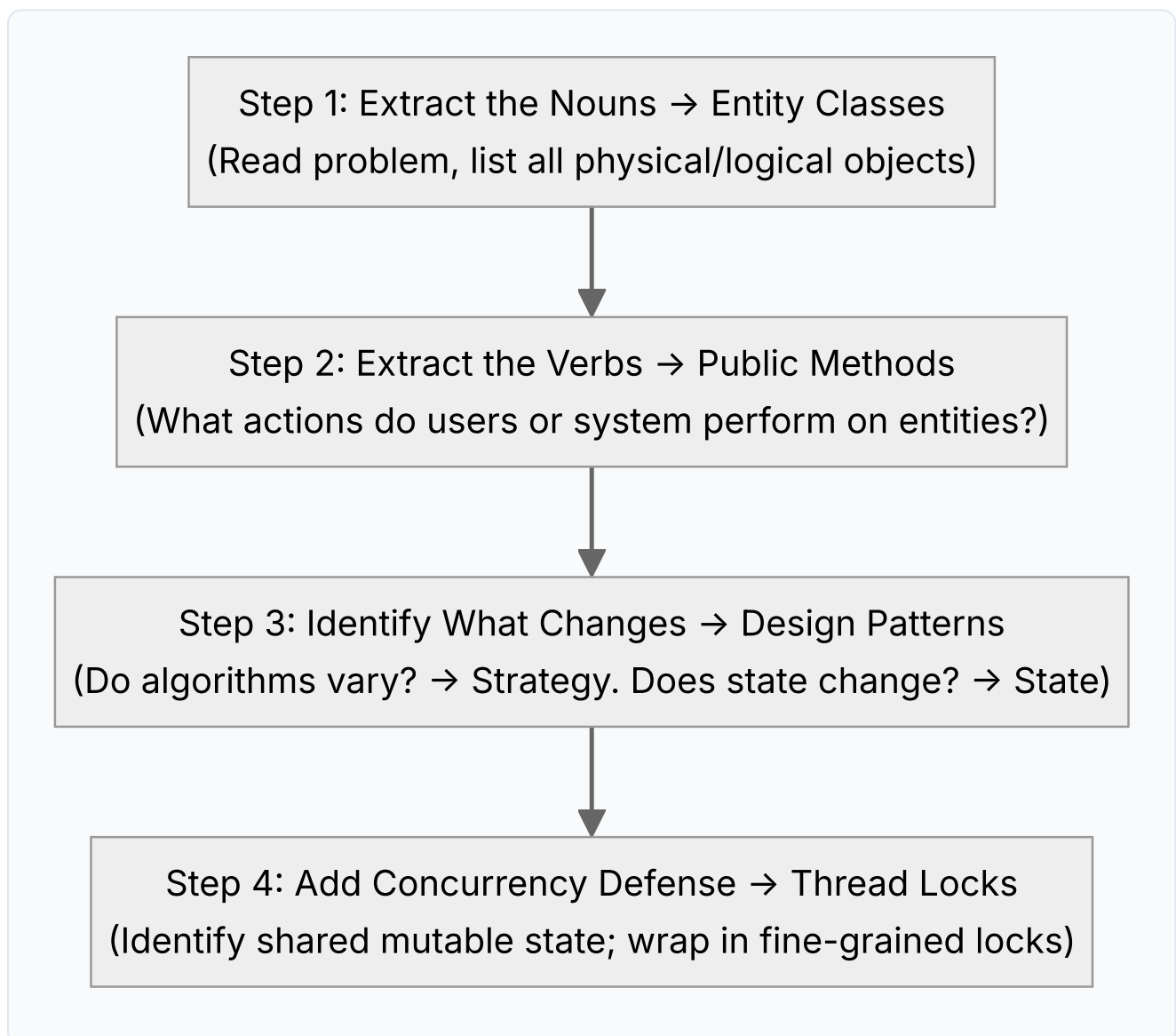


# LLD Chapter 0: The 4-Step Lego Blueprint & Pattern Decision Tree

**Core Learning Objective:** Transform Low-Level Design from an intimidating, open-ended puzzle into a calm, mechanical, 4-step assembly process. Master the visual Pattern Decision Tree to select the perfect GoF pattern for any interview question in seconds.

## 1. The 4-Step "Lego" LLD Problem-Solving Blueprint

Every Low-Level Design problem (Parking Lot, Elevator, Movie Booking, Rate Limiter) is solved using the exact same 4 mechanical steps:

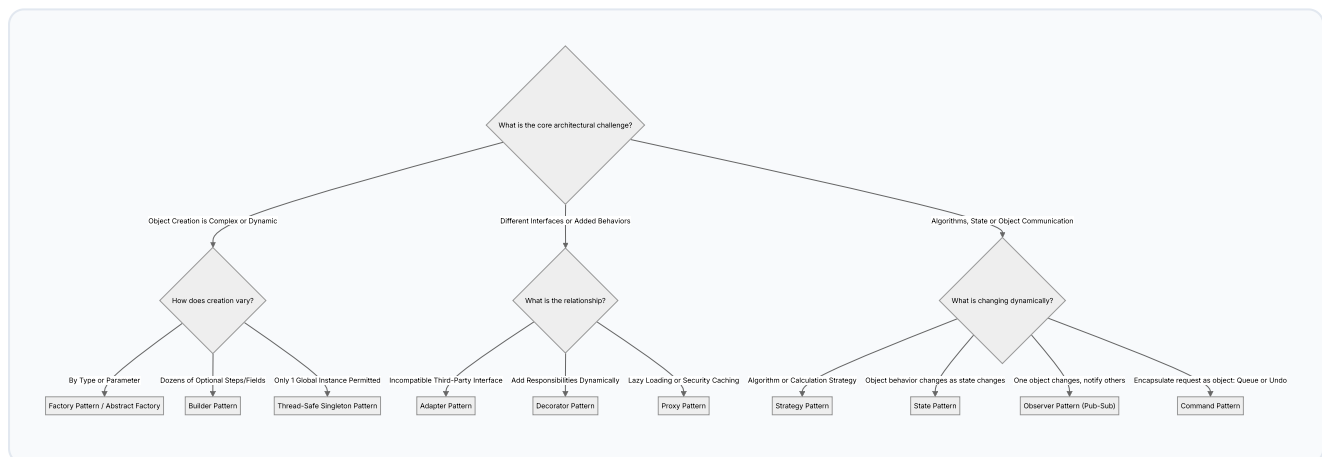


## Walkthrough on a Real Example: "Design a Parking Lot"

1. **Nouns:** Vehicle, ParkingSpot, Ticket, Payment, Floor. → *These are your classes!*
2. **Verbs:** Park, Unpark, IssueTicket, CalculateFee. → *These are your class methods!*
3. **What Changes:**
  - Spot allocation changes (Nearest vs Lowest Floor). → *Use Strategy Pattern!*
  - Vehicle types differ (Car, Bike, Truck). → *Use Factory Pattern!*
  - The parking lot configuration is a single physical facility. → *Use Singleton Pattern!*
4. **Concurrency:** Two cars attempting to grab the same spot simultaneously. → *Wrap `ParkingSpot.park()` in a `threading.Lock()` !*

That's it! In under 3 minutes, your entire architecture is completely solved before typing a line of code.

## 2. The Visual Pattern Decision Tree ("Which Pattern Do I Use When?")



## 3. The 14 Core Systems Rosetta Stone Matrix

Click on any system to jump directly into its complete, runnable Python implementation with full class hierarchies, design patterns, and thread-safety tests:

| System                                       | Primary Pattern                        | Secondary Pattern                       | The Critical Concurrency Trap                                          |
|----------------------------------------------|----------------------------------------|-----------------------------------------|------------------------------------------------------------------------|
| <a href="#">1. Distributed Job Scheduler</a> | <b>Command</b> (Jobs)                  | <b>Strategy</b> (FCFS / Priority)       | Worker race on queue; Lock during dispatch                             |
| <a href="#">2. Library Management</a>        | <b>State</b> (Available/Reserved)      | <b>Observer</b> (Hold notification)     | Race on last available physical book copy                              |
| <a href="#">3. Movie Ticket Booking</a>      | <b>State Machine</b> (Held/Booked)     | <b>Strategy</b> (Payment)               | <b>Critical:</b> 1,000 users booking seat A1; fine-grained mutex + TTL |
| <a href="#">4. Car Rental System</a>         | <b>Factory</b> (Vehicle types)         | <b>Strategy</b> (Daily / Surge fare)    | Inter-branch vehicle availability sync                                 |
| <a href="#">5. Parking Lot</a>               | <b>Singleton</b> (Facility)            | <b>Strategy</b> (Spot allocation)       | Dual cars entering gate grabbing same spot                             |
| <a href="#">6. Inventory Management</a>      | <b>Observer</b> (Low stock alerts)     | <b>Strategy</b> (Fulfillment routing)   | Overselling stock during concurrent orders                             |
| <a href="#">7. Ride Sharing (Uber)</a>       | <b>Strategy</b> (Matching / Surge)     | <b>State</b> (Trip lifecycle)           | Two drivers accepting the same ride request                            |
| <a href="#">8. Rate Limiter</a>              | <b>Strategy</b> (Token / Leaky bucket) | <b>Encapsulation</b> (Atomic counters)  | Atomic token refill under 50,000 QPS                                   |
| <a href="#">9. Snake and Ladders</a>         | <b>State</b> (Player turn queue)       | <b>Strategy</b> (Dice rolling)          | Edge overshoot past 100 win condition                                  |
| <a href="#">10. Elevator System</a>          | <b>State</b> (Up / Down / Idle)        | <b>Strategy</b> (LOOK / SCAN algorithm) | Race on hall call registration                                         |
| <a href="#">11. Vending Machine</a>          | <b>State</b> (Idle / Coin / Dispense)  | <b>State Transitions</b>                | Dispensing item without sufficient coin balance                        |
| <a href="#">12. In-Memory File System</a>    | <b>Composite</b> (Files & Dirs)        | <b>Trie</b> (Path traversal)            | Concurrency lock on parent directory path mutations                    |
| <a href="#">13. High-Throughput Logger</a>   | <b>Singleton + Observer</b>            | <b>RingBuffer</b> (Zero-lock queue)     | Producer-consumer backpressure & buffer overflow                       |
| <a href="#">14. Pub-Sub Message Broker</a>   | <b>Observer</b> (Subscribers)          | <b>Strategy</b> (Partitioning)          | Fan-out race conditions & consumer offset tracking                     |

## Further Reading

---

- [Refactoring Guru: design patterns](#)
  - [UML diagrams reference](#)
  - [Mermaid class diagrams](#)
- 

## Check Yourself

---

Answer in your head or on paper first, then open each answer.

- ▶ 1. List the steps of a 45-minute LLD interview.
- ▶ 2. How do you find classes from a problem statement?
- ▶ 3. Why ask about requirements before drawing anything?
- ▶ 4. What is a good first diagram?